

1. Which file format was smallest on your machine, and what encoding/compression characteristics help explain the result?

Parquet was by far the smallest representation at 3.28 MB, compared to CSV (27.76 MB), JSON Line (54.83 MB), and PostgreSQL (24.94 MB). Parquet achieves this through a combination of columnar storage, binary encoding, and Snappy compression. Unlike row-oriented formats (CSV and JSON), which store records sequentially, Parquet groups data by columns, allowing similar data types to be compressed together using dictionary encoding and run-length encoding.

2. Which representation was fastest for a full dataset read? Does that imply it is best for every workload?

Parquet was the fastest for a full dataset read, with a median full-read time of 0.023 seconds (compared to CSV at 0.17s, JSON at 0.51s, and PostgreSQL at 0.51s).

No. This does not imply it is best for every workload. While Parquet is exceptional for analytical queries, data warehousing scans, and batch processing, it is a poor choice for transactional workloads (OLTP) that require frequent row-level updates, row inserts, or low-latency lookups, where relational databases like PostgreSQL would excel.

3. How did filtered retrieval differ between Parquet and PostgreSQL? What additional PostgreSQL design (such as an index) could change the result?

PostgreSQL achieved a filtered query time of 0.09 seconds, while Parquet took 0.27 seconds. The PostgreSQL query performance can be significantly accelerated by creating a B-tree index or a partial or expression index on the filtered column. With an index in place, PostgreSQL can utilize index scans instead of sequential table scans, drastically reducing disk I/O and query latency for selective lookups.

4. Why is JSON Lines generally more pipeline-friendly than one giant JSON array for append/stream-oriented processing?

JSON Lines consists of independent, newline-delimited JSON objects, making it inherently stream-friendly. A giant JSON array requires the entire file to be loaded into memory simultaneously to parse the enclosing brackets and commas. In contrast, JSON Lines allows a pipeline to read, parse, process, and append records line-by-line or chunk-by-chunk in an incremental stream without consuming excessive memory or requiring the file to be completely rewritten when new records arrive.

5. What happens if a partition key has extremely high cardinality or poor query locality?

We will experience metadata overhead wherein the storage filesystem or object store gets flooded with thousands or millions of tiny directory folders and metadata files, severely degrading file listing and planning performance. Also, we will experience inefficient I/O wherein query engines spend more time opening, closing, and coordinating file handles across scattered directories rather than actually reading useful data blocks, eliminating the performance benefits of partitioning.